

ROBOTICS & SIMULATION**Theme 1: Multi-Robot Task Negotiation Engine****Problem Statement:**

Develop a decentralized coordination and mission-management platform capable of orchestrating 500+ heterogeneous autonomous mobile robots in a high-density industrial environment without depending on a continuously available central controller. The system must enable robots to reason about task ownership, negotiate shared resources, resolve right-of-way conflicts, predict collisions, detect deadlocks, and dynamically reassign work as battery levels, priorities, workloads, and robot health change. The solution should demonstrate resilient multi-agent decision-making under incomplete information and continue operating safely when individual robots or the central coordination service fail.

Key Functionalities:

- **Multi-Agent Task Allocation:** Assign and rebalance tasks using robot capability, location, workload, battery level, priority, and estimated completion time.
- **Peer-to-Peer Negotiation:** Allow robots to negotiate task ownership, shared resources, and movement priorities without a single point of decision-making.
- **Conflict & Collision Management:** Predict spatial conflicts, resolve right-of-way situations, and generate safe alternative trajectories.
- **Deadlock Detection & Recovery:** Identify cyclic waiting conditions and automatically recover through replanning, priority changes, or task migration.
- **Battery-Aware Scheduling:** Reassign missions when energy constraints threaten completion and optimize charging or return-to-base decisions.
- **Fleet Monitoring:** Provide a live simulation dashboard showing robot states, task queues, routes, conflicts, failures, and system-level performance.

Advanced Requirements:

- Continue mission execution when the central controller becomes unavailable.
- Scale the simulation from small fleets to 500+ robots and large task queues.
- Make decisions using partial, delayed, or conflicting information.
- Inject robot failures and communication loss and demonstrate graceful recovery.

Deliverables:

- Working software prototype/simulation demonstrating coordination of multiple heterogeneous robots.
- Multi-agent task allocation and peer-to-peer negotiation module.
- Collision prediction, right-of-way resolution, and deadlock detection/recovery module.
- Battery-aware task reassignment and dynamic route-replanning module.
- Failure-injection demonstration showing recovery from robot/controller failures.
- Live dashboard showing robot states, tasks, routes, conflicts, failures, and mission performance.
- Source code repository with setup/run instructions and a concise architecture document.
- Demonstration results showing scalability toward 500+ simulated robots.

Mandatory Deployment & GitHub Requirements:

- The complete application must be deployed and accessible through a working deployment URL.
- The complete source code must be pushed to a public GitHub repository before final evaluation.
- The GitHub repository must contain a README with project overview, architecture, setup instructions, deployment URL, and team/member details.
- The deployed version must match the submitted GitHub source code and be functional during evaluation.
- Teams must ensure that the deployed application can be demonstrated by the jury without requiring local setup.

Presentation & Evaluation Requirement:

Every team must be prepared to present and demonstrate their solution to the evaluators if requested. The team must be able to explain the problem approach, system architecture, key implementation decisions, working features, deployment, and results during evaluation.

Recommended Technologies:

Multi-Agent Reinforcement Learning • Game Theory • Consensus Algorithms • Auction-Based Allocation • Distributed Optimization • Conflict-Based Search

Judging Criteria:

- Distributed task allocation and negotiation effectiveness
- Collision/deadlock detection and recovery accuracy
- Route efficiency and resource utilization
- Scalability and computational efficiency
- Robustness to robot/controller failures
- Innovation in decentralized coordination
- Additional features may increase the effectiveness, usability, robustness, or overall impact of the application.

ROBOTICS & SIMULATION**Theme 2: Semantic SLAM Recovery & Map Reconstruction****Problem Statement:**

Develop a semantic mapping and localization system for autonomous robots operating in environments where visual conditions change continuously, including construction sites, industrial facilities, underground infrastructure, and disaster zones. The system must distinguish permanent structural landmarks from temporary or moving objects, quantify pose uncertainty, detect mapping drift and incorrect associations, repair corrupted map regions, reconstruct missing sections, and reconcile observations from multiple robots. GPS, beacons, and predefined landmarks must not be assumed. The final platform should expose map confidence and recovery decisions so that operators can understand where the map is reliable and where additional exploration is required.

Key Functionalities:

- **Visual & Semantic Localization:** Estimate robot pose while identifying structural features, dynamic objects, and semantic landmarks.
- **Dynamic Object Filtering:** Separate people, vehicles, temporary equipment, and other moving objects from persistent map structure.
- **Drift & Association Detection:** Detect localization drift, false loop closures, and incorrect feature-to-map associations.
- **Map Repair & Reconstruction:** Recover corrupted regions and infer missing geometry from current and historical observations.
- **Multi-Robot Map Fusion:** Merge observations from multiple robots while resolving conflicting geometry and pose estimates.
- **Confidence-Aware Mapping:** Visualize reliable, uncertain, stale, and reconstructed map regions for navigation decisions.

Advanced Requirements:

- Operate without GPS, beacons, or pre-installed landmarks.
- Handle major environmental changes such as moved machinery, blocked corridors, lighting changes, or partial destruction.
- Continuously reconcile asynchronous observations from multiple robots.
- Demonstrate recovery from deliberate mapping corruption or drift.

Deliverables:

- Working semantic SLAM/localization prototype using a provided or participant-created dataset/simulation.
- Dynamic-object detection and permanent-vs-temporary landmark classification module.
- Drift and incorrect map-association detection module.
- Map repair and missing-region reconstruction module.
- Multi-robot observation reconciliation and map-fusion module.
- Interactive map visualization showing confidence/reliability of mapped regions.
- Demonstration of recovery under environmental changes or deliberate map corruption.
- Source code, architecture documentation, dataset/source details, and evaluation results.

Mandatory Deployment & GitHub Requirements:

- The complete application must be deployed and accessible through a working deployment URL.
- The complete source code must be pushed to a public GitHub repository before final evaluation.
- The GitHub repository must contain a README with project overview, architecture, setup instructions, deployment URL, and team/member details.
- The deployed version must match the submitted GitHub source code and be functional during evaluation.
- Teams must ensure that the deployed application can be demonstrated by the jury without requiring local setup.

Presentation & Evaluation Requirement:

Every team must be prepared to present and demonstrate their solution to the evaluators if requested. The team must be able to explain the problem approach, system architecture, key implementation decisions, working features, deployment, and results during evaluation.

Recommended Technologies:

Visual SLAM • Semantic Segmentation • Feature Matching • Pose-Graph Optimization • Loop Closure • Multi-Agent Mapping

Judging Criteria:

- Localization and mapping accuracy
- Dynamic-object handling
- Drift and association-error detection

- Quality of map repair/reconstruction
- Consistency of multi-robot fusion
- Robustness under environmental change
- Innovation in semantic mapping
- Additional features may increase the effectiveness, usability, robustness, or overall impact of the application.

ROBOTICS & SIMULATION**Theme 3: Physics-Informed Drone Digital Twin****Problem Statement:**

Develop a physics-informed digital twin and autonomous decision-support simulation for a drone operating under changing environmental and physical conditions. The system must estimate the drone's evolving physical state while accounting for wind, air density, turbulence, payload, battery temperature, rotor efficiency, motor degradation, and energy consumption. Based on the estimated state and mission objectives, the digital twin should determine whether the drone can continue safely, modify its trajectory, reduce speed, alter altitude, return to base, or abort the mission. The solution should combine physics-based constraints with learned models and provide interpretable evidence for safety-critical decisions.

Key Functionalities:

- **Environmental State Model:** Simulate wind, air density, turbulence, temperature, and other environmental disturbances.
- **Vehicle Health Model:** Track battery temperature, payload changes, rotor efficiency, motor degradation, and structural stress.
- **Energy & Endurance Forecasting:** Predict power consumption, remaining endurance, and mission completion feasibility.
- **Safe Operating Envelope:** Estimate flight conditions and operating boundaries under current vehicle and environment states.
- **Autonomous Mission Decisions:** Trigger trajectory modification, speed reduction, return-to-base, or mission-abort decisions when required.
- **Digital-Twin Dashboard:** Visualize simulated telemetry, predicted states, uncertainty, environmental conditions, and control decisions.

Advanced Requirements:

- Fuse physics-based constraints with learned or data-driven models.
- Adapt to sudden wind, payload, temperature, or component-efficiency changes.
- Quantify uncertainty in state and endurance predictions.
- Demonstrate safety decisions under multiple failure and disturbance scenarios.

Deliverables:

- Working software-based drone digital-twin simulation.
- Environmental disturbance model covering wind, turbulence, air density, and related conditions.
- Vehicle-health model covering battery temperature, payload, rotor efficiency, and motor degradation.
- Energy-consumption, endurance, and safe-operating-envelope prediction module.
- Autonomous decision engine for trajectory modification, speed reduction, return-to-base, or mission abort.
- Dashboard displaying simulated telemetry, physical state, uncertainty, environmental conditions, and decisions.
- Scenario demonstrations covering sudden environmental and vehicle-state changes.
- Source code, model documentation, assumptions, and performance/evaluation report.

Mandatory Deployment & GitHub Requirements:

- The complete application must be deployed and accessible through a working deployment URL.
- The complete source code must be pushed to a public GitHub repository before final evaluation.
- The GitHub repository must contain a README with project overview, architecture, setup instructions, deployment URL, and team/member details.
- The deployed version must match the submitted GitHub source code and be functional during evaluation.
- Teams must ensure that the deployed application can be demonstrated by the jury without requiring local setup.

Presentation & Evaluation Requirement:

Every team must be prepared to present and demonstrate their solution to the evaluators if requested. The team must be able to explain the problem approach, system architecture, key implementation decisions, working features, deployment, and results during evaluation.

Recommended Technologies:

Physics-Informed Neural Networks • Neural ODEs • Model Predictive Control • Kalman Filtering • Reinforcement Learning • Digital Twins

Judging Criteria:

- Physical-state estimation accuracy
- Environmental and degradation modelling quality
- Energy/endurance prediction accuracy

- Effectiveness of safety and control decisions
- Robustness under changing conditions
- Physics/ML integration
- Innovation in digital-twin design
- Additional features may increase the effectiveness, usability, robustness, or overall impact of the application.

ROBOTICS & SIMULATION**Theme 4: Robot Fleet Recovery Under Cascading Failures****Problem Statement:**

Develop an intelligent fleet-management and recovery platform that maintains the maximum achievable mission performance when failures propagate across a fleet of hundreds of robots. The system must model robot health, identify mission-critical assets, predict how one failure may affect dependent tasks or neighbouring robots, migrate work to available resources, rebalance the fleet, and optimize remaining battery and capacity. Participants should build a simulation in which failures can occur in arbitrary sequences rather than only in predefined scenarios. The platform should quantify mission degradation and demonstrate graceful recovery while avoiding unstable reassignment loops or excessive resource consumption.

Key Functionalities:

- **Robot Health & Failure Prediction:** Estimate robot health and predict likely failures from simulated telemetry and historical behaviour.
- **Failure-Propagation Analysis:** Model task, dependency, communication, and capacity relationships to identify cascading impact.
- **Dynamic Task Migration:** Move interrupted or at-risk missions to suitable robots while respecting deadlines and capabilities.
- **Fleet Rebalancing:** Continuously rebalance workload, battery availability, and robot capacity after failures.
- **Mission-Criticality Engine:** Prioritize recovery of tasks based on deadlines, dependencies, operational value, and resource needs.
- **Recovery Command Center:** Display fleet state, failure chains, migrated tasks, performance loss, and recovery actions.

Advanced Requirements:

- Handle previously unseen sequences and combinations of failures.
- Maintain the highest achievable mission completion rate under degraded fleet capacity.
- Avoid excessive task migration and unnecessary battery consumption.
- Support progressive failure injection during an active mission.

Deliverables:

- Working fleet-management simulation with configurable robot health and mission states.
- Robot-health and failure-prediction module.
- Failure-propagation graph/model identifying affected tasks and resources.
- Dynamic task-migration and fleet-rebalancing engine.
- Mission-criticality and prioritization mechanism.
- Failure-injection scenarios demonstrating recovery from previously unseen failure sequences.
- Command-center dashboard showing failures, recovery actions, task migration, and mission performance.
- Source code, architecture documentation, test scenarios, and quantitative recovery results.

Mandatory Deployment & GitHub Requirements:

- The complete application must be deployed and accessible through a working deployment URL.
- The complete source code must be pushed to a public GitHub repository before final evaluation.
- The GitHub repository must contain a README with project overview, architecture, setup instructions, deployment URL, and team/member details.
- The deployed version must match the submitted GitHub source code and be functional during evaluation.
- Teams must ensure that the deployed application can be demonstrated by the jury without requiring local setup.

Presentation & Evaluation Requirement:

Every team must be prepared to present and demonstrate their solution to the evaluators if requested. The team must be able to explain the problem approach, system architecture, key implementation decisions, working features, deployment, and results during evaluation.

Recommended Technologies:

Distributed Systems • Reliability Engineering • Predictive Analytics • Optimization • Graph-Based Failure Analysis • Fleet Management

Judging Criteria:

- Failure prediction accuracy
- Failure-propagation analysis
- Task migration and fleet-rebalancing efficiency

- Mission-performance preservation
- Resource/battery optimization
- Robustness under cascading failures
- Innovation in recovery strategy
- Additional features may increase the effectiveness, usability, robustness, or overall impact of the application.

CYBERSECURITY***Theme 5: Zero-Trust Agent Identity & Privilege Fabric*****Problem Statement:**

Develop a continuous, context-aware authorization platform for AI agents, developers, services, and robots accessing critical digital resources. Instead of relying on static roles or long-lived credentials, the system should continuously evaluate identity, device state, task context, historical behaviour, resource sensitivity, network signals, and current risk. It must issue short-lived, context-bound credentials and dynamically decide whether access should continue, be restricted, require re-authentication, have privileges reduced, or be revoked. The platform should also calculate the potential blast radius of a compromised identity and generate an appropriate isolation strategy.

Key Functionalities:

- **Identity & Device Trust:** Evaluate user, agent, service, robot, device posture, authentication state, and session context.
- **Behavioural Risk Engine:** Compare current activity with historical behaviour and detect anomalous access patterns.
- **Resource Sensitivity:** Classify resources by business or operational criticality and incorporate sensitivity into access decisions.
- **Ephemeral Credential Service:** Issue short-lived credentials bound to identity, device, task, resource, and risk context.
- **Continuous Authorization:** Re-evaluate access throughout a session and dynamically reduce, extend, or revoke privileges.
- **Compromise Isolation:** Estimate blast radius and recommend or simulate account, device, service, or network isolation.

Advanced Requirements:

- Update access decisions as risk changes instead of relying only on login-time checks.
- Model compromised identities and determine reachable resources and privilege paths.
- Demonstrate containment without shutting down unaffected users and services.
- Maintain auditable decision logs explaining why access changed.

Deliverables:

- Working zero-trust authorization prototype for users, AI agents, services, or robots.
- Identity/device-state and behavioural risk evaluation engine.
- Resource-sensitivity and context-aware authorization module.
- Short-lived, context-bound credential issuance mechanism.
- Continuous authorization engine capable of reducing, extending, or revoking privileges.
- Compromised-identity blast-radius analysis and isolation demonstration.
- Auditable decision logs explaining access decisions and changes.
- Source code, architecture/security design, threat scenarios, and evaluation results.

Mandatory Deployment & GitHub Requirements:

- The complete application must be deployed and accessible through a working deployment URL.
- The complete source code must be pushed to a public GitHub repository before final evaluation.
- The GitHub repository must contain a README with project overview, architecture, setup instructions, deployment URL, and team/member details.
- The deployed version must match the submitted GitHub source code and be functional during evaluation.
- Teams must ensure that the deployed application can be demonstrated by the jury without requiring local setup.

Presentation & Evaluation Requirement:

Every team must be prepared to present and demonstrate their solution to the evaluators if requested. The team must be able to explain the problem approach, system architecture, key implementation decisions, working features, deployment, and results during evaluation.

Recommended Technologies:

Zero Trust Architecture • Continuous Authentication • Graph-Based Authorization • Behavioural Analytics • Ephemeral Credentials • Risk-Adaptive Access Control

Judging Criteria:

- Identity/risk assessment accuracy
- Continuous authorization effectiveness
- Credential-management security

- Privilege-decision accuracy
- Compromised-identity isolation
- Adaptability to changing risk
- Innovation in zero-trust implementation
- Additional features may increase the effectiveness, usability, robustness, or overall impact of the application.

CYBERSECURITY**Theme 6: Software Supply-Chain Attack Graph Engine****Problem Statement:**

Develop a continuously updated software supply-chain security analysis platform that covers source repositories, dependencies, packages, build systems, containers, APIs, and CI/CD pipelines. The engine must correlate dependency relationships, source-code behaviour, version changes, maintainer activity, build metadata, and simulated runtime observations to identify malicious or compromised components. Beyond detecting a vulnerable package, the system should trace how a compromise can propagate through direct and transitive dependencies, determine affected assets, estimate confidence and business impact, identify the likely attack origin, and recommend containment actions.

Key Functionalities:

- **Dependency & Asset Graph:** Build an interconnected graph of repositories, packages, versions, services, containers, APIs, and deployments.
- **Malicious Component Detection:** Detect typosquatting, dependency confusion, suspicious updates, dormant logic, and obfuscated behaviour.
- **Code & Behaviour Analysis:** Combine static analysis, code embeddings, metadata, and simulated runtime signals.
- **Attack-Path Tracing:** Trace compromise from suspected origin through direct and transitive dependencies to affected assets.
- **Impact & Confidence Engine:** Estimate exploitability, blast radius, confidence, and operational impact.
- **Containment Recommendations:** Recommend package pinning, rollback, isolation, dependency replacement, or pipeline controls.

Advanced Requirements:

- Detect threats hidden several dependency levels away from the primary application.
- Correlate source, build, dependency, and runtime evidence rather than using a single signal.
- Explain why a component is considered suspicious and show the propagation path.
- Support continuous re-analysis when package versions or build metadata change.

Deliverables:

- Working software supply-chain analysis platform.
- Dependency and asset graph covering repositories, packages, versions, services, containers, APIs, and pipelines.
- Detection modules for suspicious package behaviour, typosquatting, dependency confusion, and compromised updates.
- Static/code/behaviour analysis pipeline using the selected technical approach.
- Attack-origin and propagation-path visualization.
- Impact, confidence, affected-asset, and containment recommendation engine.
- Dashboard demonstrating continuous re-analysis when dependencies or metadata change.
- Source code, sample attack scenarios, architecture documentation, and detection/evaluation report.

Mandatory Deployment & GitHub Requirements:

- The complete application must be deployed and accessible through a working deployment URL.
- The complete source code must be pushed to a public GitHub repository before final evaluation.
- The GitHub repository must contain a README with project overview, architecture, setup instructions, deployment URL, and team/member details.
- The deployed version must match the submitted GitHub source code and be functional during evaluation.
- Teams must ensure that the deployed application can be demonstrated by the jury without requiring local setup.

Presentation & Evaluation Requirement:

Every team must be prepared to present and demonstrate their solution to the evaluators if requested. The team must be able to explain the problem approach, system architecture, key implementation decisions, working features, deployment, and results during evaluation.

Recommended Technologies:

AST Analysis • Code Embeddings • Dependency Graphs • Graph Neural Networks • Static Analysis • Behavioural Anomaly Detection

Judging Criteria:

- Malicious-component detection accuracy
- Dependency/attack-path analysis
- Transitive-threat detection

- Origin and propagation tracing
- Impact/confidence assessment
- Containment recommendations
- Innovation in supply-chain security
- Additional features may increase the effectiveness, usability, robustness, or overall impact of the application.

CYBERSECURITY**Theme 7: Privacy-Preserving Threat Intelligence Network****Problem Statement:**

Develop a federated threat-intelligence platform that enables multiple organizations to collaboratively improve detection of emerging cyberattacks without sharing their raw security telemetry. Each participant should retain control of its local logs or security data while contributing privacy-preserving model updates to a shared intelligence layer. The system must protect sensitive organizational information, resist model-poisoning attempts, identify malicious participants, and remain useful when some participants are unreliable or compromised. Teams should demonstrate that collaborative detection can improve while privacy leakage, communication overhead, and adversarial manipulation are explicitly measured.

Key Functionalities:

- **Federated Threat Learning:** Train a shared threat-detection model from distributed organizational datasets without centralizing raw telemetry.
- **Secure Aggregation:** Combine model updates so individual participant contributions cannot be trivially reconstructed.
- **Privacy Protection:** Apply differential privacy or other mechanisms to reduce leakage from shared updates.
- **Poisoning & Sybil Detection:** Identify anomalous, malicious, or coordinated participants attempting to corrupt the global model.
- **Threat Intelligence Fusion:** Convert model outputs and local indicators into shared, confidence-aware intelligence.
- **Federated Operations Dashboard:** Show participating nodes, model rounds, detection metrics, privacy settings, and adversarial events.

Advanced Requirements:

- Detect and contain participants attempting to poison the shared model.
- Demonstrate resilience when a subset of participants provides manipulated updates.
- Protect sensitive organizational information while maintaining useful detection quality.
- Measure the trade-off between privacy, accuracy, communication cost, and training time.

Deliverables:

- Working federated threat-intelligence prototype with multiple simulated organizations.
- Federated learning/training pipeline without centralizing raw organizational telemetry.
- Secure aggregation and privacy-preservation mechanism.
- Malicious-participant and model-poisoning detection/containment mechanism.
- Shared threat-intelligence output with confidence indicators.
- Dashboard showing federation rounds, participants, detection performance, privacy settings, and attacks.
- Comparison of detection accuracy, privacy protection, communication cost, and computational overhead.
- Source code, security/privacy design, experiment configuration, and evaluation report.

Mandatory Deployment & GitHub Requirements:

- The complete application must be deployed and accessible through a working deployment URL.
- The complete source code must be pushed to a public GitHub repository before final evaluation.
- The GitHub repository must contain a README with project overview, architecture, setup instructions, deployment URL, and team/member details.
- The deployed version must match the submitted GitHub source code and be functional during evaluation.
- Teams must ensure that the deployed application can be demonstrated by the jury without requiring local setup.

Presentation & Evaluation Requirement:

Every team must be prepared to present and demonstrate their solution to the evaluators if requested. The team must be able to explain the problem approach, system architecture, key implementation decisions, working features, deployment, and results during evaluation.

Recommended Technologies:

Federated Learning • Secure Multiparty Computation • Differential Privacy • Homomorphic Encryption • Anomaly Detection • Cryptography

Judging Criteria:

- Collaborative detection effectiveness
- Strength of privacy protection
- Malicious-participant detection

- Resistance to model poisoning
- Security of aggregation
- Communication/computational efficiency
- Innovation in privacy-preserving cybersecurity
- Additional features may increase the effectiveness, usability, robustness, or overall impact of the application.

AI/ML REASONING**Theme 8: Multi-Agent AI Reasoning & Verification Engine****Problem Statement:**

Develop a multi-agent AI reasoning platform in which specialized agents collaborate to solve complex tasks while every important output is independently verified before acceptance. The system should separate generation from verification and use evidence, structured checks, tool execution, and contradiction analysis to identify hallucinations, invalid reasoning, unsupported claims, incorrect API usage, unsafe actions, and low-confidence conclusions. When a failure is detected, the system should route the task back for correction, request additional evidence, or reject the task when reliable completion is impossible. The platform must remain dependable when information is incomplete, ambiguous, conflicting, or deliberately misleading.

Key Functionalities:

- **Specialized Agent Orchestration:** Coordinate planner, researcher, coder/tool-use, verifier, critic, and finalizer agents with explicit responsibilities.
- **Evidence Retrieval & Grounding:** Retrieve relevant evidence and link claims to supporting sources or structured data.
- **Independent Verification:** Use separate verification paths to test factual claims, calculations, code/API usage, and logical consistency.
- **Contradiction & Risk Detection:** Detect conflicting evidence, unsupported claims, unsafe actions, and low-confidence conclusions.
- **Self-Correction Loop:** Return failed outputs to appropriate agents, track revisions, and re-verify before acceptance.
- **Decision & Audit Layer:** Expose evidence, confidence, agent decisions, verification results, and rejection reasons.

Advanced Requirements:

- Handle ambiguous tasks, incomplete data, conflicting sources, misleading documents, and invalid APIs.
- Recognize when evidence is insufficient and explicitly decline unreliable conclusions.
- Use sandboxed execution for code or tool verification where appropriate.
- Measure verification quality separately from generation quality.

Deliverables:

- Working multi-agent AI orchestration prototype with clearly separated agent responsibilities.
- Evidence retrieval/grounding module for supporting important claims.
- Independent verification layer for factual, logical, computational, or tool-use outputs.
- Contradiction, hallucination, unsupported-claim, and unsafe-action detection module.
- Self-correction and re-verification workflow.
- Decision/audit interface showing evidence, confidence, verification results, revisions, and rejection reasons.
- Evaluation set covering ambiguous, incomplete, conflicting, and misleading inputs.
- Source code, agent architecture/prompt documentation, evaluation metrics, and demonstration results.

Mandatory Deployment & GitHub Requirements:

- The complete application must be deployed and accessible through a working deployment URL.
- The complete source code must be pushed to a public GitHub repository before final evaluation.
- The GitHub repository must contain a README with project overview, architecture, setup instructions, deployment URL, and team/member details.
- The deployed version must match the submitted GitHub source code and be functional during evaluation.
- Teams must ensure that the deployed application can be demonstrated by the jury without requiring local setup.

Presentation & Evaluation Requirement:

Every team must be prepared to present and demonstrate their solution to the evaluators if requested. The team must be able to explain the problem approach, system architecture, key implementation decisions, working features, deployment, and results during evaluation.

Recommended Technologies:

Multi-Agent Orchestration • Retrieval-Augmented Verification • Sandboxed Execution • Structured Validation • Self-Correction Loops • Evaluation Metrics

Judging Criteria:

- Hallucination/error detection accuracy
- Independent verification effectiveness

- Evidence-grounded reasoning quality
- Contradiction/unsupported-claim detection
- Self-correction effectiveness
- Reliability under ambiguity
- Innovation in multi-agent reasoning
- Additional features may increase the effectiveness, usability, robustness, or overall impact of the application.